

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

ОТЧЕТ ПО Лабораторной работе №1
по дисциплине Программирование микроконтроллеров

Создание нового проекта в Keil uVision 5

Студент,
группы 5130201/40003

_____ Адиатуллин Т.Р

Руководитель,
Преподаватель

_____ Вербова Н. М.

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

1	Цель работы	3
2	Постановка задачи	3
3	Алгоритм работы программы	3
3.1	Инициализация проекта	3
3.2	Настройка портов GPIO	3
3.3	Реализация мигания светодиодов	4
	Заключение	8

1. Цель работы

Ознакомиться с основными приемами работы с документацией при составлении программ для микроконтроллеров

2. Постановка задачи

Создать новый проект в Keil μ Vision5 и разработать программу для микроконтроллера STM32F200, которая включает и выключает светодиод, а также вторую часть задания с “бегущим светодиодом”, а именно последовательного включения всех светодиодов.

3. Алгоритм работы программы

Данный раздел описывает последовательность действий, необходимых для разработки и реализации проекта на базе микроконтроллера STM32F207IGHx в среде Keil μ Vision5. Алгоритм включает этапы инициализации проекта, настройки портов ввода–вывода, реализации мигания светодиода и модификации программы для создания эффекта «бегущего огня».

3.1. Инициализация проекта

1. Создать новый проект в среде Keil μ Vision5.
2. Выбрать микроконтроллер семейства STM32F207IGHx.
3. Выполнить базовую конфигурацию проекта:
 - подключить поддержку ядра CMSIS (CMSIS CORE);
 - добавить стартовый файл;
 - подключить необходимые модули HAL.

3.2. Настройка портов GPIO

Для работы со светодиодами требуется настроить порты GPIOG, GPION и GPIOI.

1. Разрешить тактирование соответствующего порта, установив нужные биты в регистре RCC_AHB1ENR.
2. Настроить режим работы выводов как «выход» через регистр GPIOx_MODER.
3. Определить параметры выхода:

- тип выхода — push–pull (GPIOx_OTYPER);
- низкая скорость переключения (GPIOx_OSPEEDR).

3.3. Реализация мигания светодиодов

1. Включить тактирование портов ввода-вывода (GPIOG, GPION, GPIOI), установив соответствующие биты в регистре управления тактированием (по адресу 0x40023830).
2. Настроить используемые пины (PG6, PG7, PG8, PH2, PH3, PH6, PH7, PI10) на режим вывода (general purpose output mode), записав значение 01 в соответствующие биты регистров MODER каждого порта.
3. Установить бит нужного пина (начиная с PH3) в регистре выходных данных (ODR) в состояние «1», включая светодиод.
4. Выполнить пустой цикл (for) для создания программной задержки, необходимой для визуального восприятия.
5. Сбросить бит этого же пина в регистре ODR в «0», выключив светодиод.
6. Повторить шаги 3–5 для всех остальных задействованных пинов в заданной последовательности для получения эффекта поочередного мигания.

Код программы

```
1 int main()
2 {
3     int i;
4     unsigned long int j;
5     i = 0;
6     j = 0;
7
8     // 1. Включение тактирования портов вводавывода- регистр (
9     RCC_AHB1ENR, адрес 0x40023830)
10    // Чтобы включить тактирование порта, нужно установить
11    // соответствующий бит в 1.
12    // Вычисление значения: 2 в степени номер ( бита). Перевод в
13    // рич16- систему:
14    // Бит 6 порт ( G) = 2^6 = 64 = 0x40
15    // Бит 7 порт ( H) = 2^7 = 128 = 0x80
16    // Бит 8 порт ( I) = 2^8 = 256 = 0x100
17    *(unsigned long*)(0x40023830) |= 0x40; // Включаем тактирование
18    GPIOG
19    *(unsigned long*)(0x40023830) |= 0x80; // Включаем тактирование
20    GPIOH
21    *(unsigned long*)(0x40023830) |= 0x100; // Включаем тактирование
22    GPIOI
23
24    // Программная задержка для стабилизации частоты перед настройкой
25    // портов
26    for (i = 0; i < 4; i++) {}
27
28    // 2. Настройка направления работы пинов регистры ( MODER)
29    // В регистре MODER на каждый пин выделяется по 2 бита с (
30    // номерами 2*N и 2*N + 1, где N - номер пина).
31    // Для режима выхода (Output) нужно записать комбинацию "01", то
32    // есть:
33    // - сбросить старший бит (2*N + 1) в 0 через ( операцию И с
34    // инвертированной маской)
35    // - установить младший бит (2*N) в 1 через ( операцию ИЛИ)
36    // Пример расчета для пина 6: нужные биты 13 и 12.
37    // 2^13 = 8192 = 0x2000 маска ( для сброса). 2^12 = 4096 = 0x1000
38    // маска ( для установки).
39
40    // Порт G базовый ( адрес 0x40021800)
41    *(unsigned long*)(0x40021800) = (*(unsigned long*)(0x40021800) &
42    (~0x00002000)) | (0x00001000);
43    // PG6: сбрасываем бит 13 (0x2000) и 12 (0x1000)
44
45    *(unsigned long*)(0x40021800) = (*(unsigned long*)(0x40021800) &
46    (~0x00008000)) | (0x00004000);
47    // PG7: биты 15 и 14. 2^15 = 0x8000 сброс (), 2^14 = 0x4000
48    установка ()
```

```

36 * (unsigned long*) (0x40021800) = (* (unsigned long*) (0x40021800) &
    (~0x00020000)) | (0x00010000);
37 // PG8: биты 17 и 16. 2^17 = 0x20000 сброс(), 2^16 = 0x10000
    установка()
38
39 // Порт H базовый( адрес 0x40021C00)
40 * (unsigned long*) (0x40021C00) = (* (unsigned long*) (0x40021C00) &
    (~0x00000020)) | (0x00000010);
41 // PH2: биты 5 и 4. 2^5 = 0x20 сброс(), 2^4 = 0x10 установка()
42
43 * (unsigned long*) (0x40021C00) = (* (unsigned long*) (0x40021C00) &
    (~0x00000080)) | (0x00000040);
44 // PH3: биты 7 и 6. 2^7 = 0x80 сброс(), 2^6 = 0x40 установка()
45
46 * (unsigned long*) (0x40021C00) = (* (unsigned long*) (0x40021C00) &
    (~0x00002000)) | (0x00001000);
47 // PH6: биты 13 и 12. 2^13 = 0x2000 сброс(), 2^12 = 0x1000
    установка()
48
49 * (unsigned long*) (0x40021C00) = (* (unsigned long*) (0x40021C00) &
    (~0x00008000)) | (0x00004000);
50 // PH7: биты 15 и 14. 2^15 = 0x8000 сброс(), 2^14 = 0x4000
    установка()
51
52 // Порт I базовый( адрес 0x40022000)
53 * (unsigned long*) (0x40022000) = (* (unsigned long*) (0x40022000) &
    (~0x00200000)) | (0x00100000);
54 // PI10: биты 21 и 20. 2^21 = 0x200000 сброс(), 2^20 = 0x100000
    установка()
55
56 // 3. Управление светодиодами через регистры ODR смещение (+0x14
    от базы порта)
57 // Здесь каждый пин занимает ровно 1 бит совпадает( с номером
    пина N).
58 // Значение маски рассчитывается как 2^N. Установка бита (|=)
    включает диод, сброс (&= ~) выключает.
59 while(1) {
60     // PH3: бит 3 -> 2^3 = 8 = 0x08
61     * (unsigned long*) (0x40021C14) |= 0x08; // Включение
62     for (j = 0; j < 1000000; j++) {} // Задержка для
    видимости эффекта
63     * (unsigned long*) (0x40021C14) &= ~0x08; // Выключение
64
65     // PH6: бит 6 -> 2^6 = 64 = 0x40
66     * (unsigned long*) (0x40021C14) |= 0x40; // Включение
67     for (j = 0; j < 1000000; j++) {} // Задержка для видимости
    эффекта
68     * (unsigned long*) (0x40021C14) &= ~0x40; // Выключение
69
70     // PH7: бит 7 -> 2^7 = 128 = 0x80
71     * (unsigned long*) (0x40021C14) |= 0x80; // Включение
72     for (j = 0; j < 1000000; j++) {} // Задержка для видимости
    эффекта

```

```

73  * (unsigned long*) (0x40021C14) &= ~0x80;  // Выключение
74
75  // PI10: бит 10 -> 2^10 = 1024 = 0x400
76  * (unsigned long*) (0x40022014) |= 0x400;  // Включение
77  for (j = 0; j < 1000000; j++) {} // Задержка для видимости
эффекта
78  * (unsigned long*) (0x40022014) &= ~0x400;  // Выключение
79
80  // PG6: бит 6 -> 2^6 = 64 = 0x40
81  * (unsigned long*) (0x40021814) |= 0x40;  // Включение
82  for (j = 0; j < 1000000; j++) {} // Задержка для видимости
эффекта
83  * (unsigned long*) (0x40021814) &= ~0x40;  // Выключение
84
85  // PG7: бит 7 -> 2^7 = 128 = 0x80
86  * (unsigned long*) (0x40021814) |= 0x80;  // Включение
87  for (j = 0; j < 1000000; j++) {} // Задержка для видимости
эффекта
88  * (unsigned long*) (0x40021814) &= ~0x80;  // Выключение
89
90  // PG8: бит 8 -> 2^8 = 256 = 0x100
91  * (unsigned long*) (0x40021814) |= 0x100;  // Включение
92  for (j = 0; j < 1000000; j++) {} // Задержка для видимости
эффекта
93  * (unsigned long*) (0x40021814) &= ~0x100;  // Выключение
94
95  // PH2: бит 2 -> 2^2 = 4 = 0x04
96  * (unsigned long*) (0x40021C14) |= 0x04;  // Включение
97  for (j = 0; j < 1000000; j++) {} // Задержка для видимости
эффекта
98  * (unsigned long*) (0x40021C14) &= ~0x04;  // Выключение
99
100 // PG8: снова бит 8 -> 0x100
101 * (unsigned long*) (0x40021814) |= 0x100;  // Включение
102 for (j = 0; j < 1000000; j++) {} // Задержка для видимости
эффекта
103 * (unsigned long*) (0x40021814) &= ~0x100;  // Выключение
104 }
105 }

```

Listing 1: Основная функция программы

Заключение

В ходе лабораторной работы у меня получилось ознакомиться с основными приемами работы с документацией при составлении программ для микроконтроллеров. Я научилась создавать новый проект в Keil uVision5 и разработала программу для микроконтроллера (МК) STM32F200, которая включает и выключает светодиод. Также была выполнена модификация задания: программа, которая последовательно включает и выключает каждый из 8 светодиодов.